

Язык программирования C++

Лекция 15: статические методы, паттерны проектирования, синглтоны, зависимости, автотестирование

Никита Лисица

2026

Inline методы класса

- Напоминание: метод, определённый (реализованный) прямо в определении класса, по умолчанию является **inline**

```
class Array {  
private:  
    Object* data_;  
    size_t size_;  
  
public:  
    // Автоматически inline  
    size_t size() const {  
        return size_;  
    }  
}
```

Inline методы класса

- Метод, определённый снаружи класса, тоже можно сделать **inline**:

```
class Array {  
private:  
    Object* data_;  
    size_t size_;  
  
public:  
    // Не будет автоматически  
    // помечен как inline  
    size_t size() const;  
}  
  
// Но можно написать явно!  
inline size_t Array::size() const {  
    return size_;  
}
```

Статические переменные

- Напоминание: статическая переменная внутри функции 'живёт' (т.е. сохраняет значение, у неё не вызывается деструктор) и между вызовами функции:

```
int gen_unique_id() {  
    static int next_id = 0;  
    return next_id++;  
}
```

Статические переменные

- Напоминание: статическая переменная внутри функции 'живёт' (т.е. сохраняет значение, у неё не вызывается деструктор) и между вызовами функции:

```
int gen_unique_id() {  
    static int next_id = 0;  
    return next_id++;  
}
```

- В каком-то смысле, такая переменная одна на все вызовы функции (т.е. не создаётся каждый раз)

Статические поля класса

- Поле класса тоже может быть статическим – тогда оно одно на все объекты (экземпляры) класса:

```
// object.h

class Object {
private:
    // Объявление статического поля
    static int next_id;
    int id;

public:
    Object()
        : id(next_id++)
    {}
};
```

Статические поля класса

- Статическое поле является, по сути, глобальной переменной, привязанной к классу

Статические поля класса

- Статическое поле является, по сути, глобальной переменной, привязанной к классу
- $\implies$ Под неё нужно выделить место в каком-нибудь `cpp`-файле:

```
// object.cpp  
  
// Определение статического поля  
int Object::next_id = 0;
```


Статические поля класса

- Статическое поле является, по сути, глобальной переменной, привязанной к классу
- $\implies$ Под неё нужно выделить место в каком-нибудь `cpp`-файле:

```
// object.cpp

// Определение статического поля
int Object::next_id = 0;
```

- Использование:

```
// main.cpp
#include <object.h>

int main() {
    Object o1; // o1.id == 0
    Object o2; // o2.id == 1
}
```

Статические методы класса

- Метод класса тоже может быть статическим

Статические методы класса

- Метод класса тоже может быть статическим
- Он тоже 'один на все объекты класса': у него нет **this**, и он не имеет доступа к нестатическим полям класса:

```
class Object {
private:
    static size_t count_;
    const char* name_;

public:
    Object(const char* name)
        : name_(name)
    {
        ++count_;
    }

    static size_t count() {
        // Нет доступа к this или name_
        return count_;
    }
};
```

- Статические методы можно вызывать как через объект класса, так и через сам класс:

```
int main() {  
    Object o1;  
  
    // Эти два вызова эквивалентны,  
    // есл метод статический:  
    o1.count();  
    Object::count();  
}
```

Паттерны (шаблоны) проектирования

- Повторяющиеся и переиспользуемые архитектурные конструкции, решающие определённые архитектурные задачи

Паттерны (шаблоны) проектирования

- Повторяющиеся и переиспользуемые архитектурные конструкции, решающие определённые архитектурные задачи
- Популярные книги по теме:
 - E. Gamma, R. Helm, R. Johnson, J. Vlissides – *Design Patterns: Elements of Reusable Object-Oriented Software*
 - R. Martin – *Clean Code: A Handbook of Agile Software Craftsmanship*
 - R. Nystrom – *Game Programming Patterns*

Паттерны (шаблоны) проектирования

- Повторяющиеся и переиспользуемые архитектурные конструкции, решающие определённые архитектурные задачи
- Популярные книги по теме:
 - E. Gamma, R. Helm, R. Johnson, J. Vlissides – *Design Patterns: Elements of Reusable Object-Oriented Software*
 - R. Martin – *Clean Code: A Handbook of Agile Software Craftsmanship*
 - R. Nystrom – *Game Programming Patterns*
- Все эти паттерны – *советы, а не строгие правила*, и не стоит пытаться реализовать всё через них

Паттерны (шаблоны) проектирования

- Повторяющиеся и переиспользуемые архитектурные конструкции, решающие определённые архитектурные задачи
- Популярные книги по теме:
 - E. Gamma, R. Helm, R. Johnson, J. Vlissides – *Design Patterns: Elements of Reusable Object-Oriented Software*
 - R. Martin – *Clean Code: A Handbook of Agile Software Craftsmanship*
 - R. Nystrom – *Game Programming Patterns*
- Все эти паттерны – *советы, а не строгие правила*, и не стоит пытаться реализовать всё через них

```
R"(
    Sometimes, the elegant implementation is just a function.
    Not a method. Not a class. Not a framework.
    Just a function.

    - John Carmack
)"
```


Паттерн синглтон (singleton)

- Хотим, чтобы в программе гарантированно был только один объект конкретного класса

Паттерн синглтон (singleton)

- Хотим, чтобы в программе гарантированно был только один объект конкретного класса
- Попытка создать два объекта должна быть невозможна (ошибка компиляции)

Паттерн синглтон (singleton)

- Хотим, чтобы в программе гарантированно был только один объект конкретного класса
- Попытка создать два объекта должна быть невозможна (ошибка компиляции)
- Примеры:

Паттерн синглтон (singleton)

- Хотим, чтобы в программе гарантированно был только один объект конкретного класса
- Попытка создать два объекта должна быть невозможна (ошибка компиляции)
- Примеры:
 - Класс глобальных настроек

Паттерн синглтон (singleton)

- Хотим, чтобы в программе гарантированно был только один объект конкретного класса
- Попытка создать два объекта должна быть невозможна (ошибка компиляции)
- Примеры:
 - Класс глобальных настроек
 - Система логирования

Паттерн синглтон (singleton)

- Хотим, чтобы в программе гарантированно был только один объект конкретного класса
- Попытка создать два объекта должна быть невозможна (ошибка компиляции)
- Примеры:
 - Класс глобальных настроек
 - Система логирования
 - Общий ресурс, используемый разными частями программы

Паттерн синглтон (singleton)

- Нельзя позволять пользователю самому создавать объект $\Rightarrow$ конструктор должен быть приватным, копирование нужно запретить

Паттерн синглтон (singleton)

- Нельзя позволять пользователю самому создавать объект $\Rightarrow$ конструктор должен быть приватным, копирование нужно запретить
- Сам единственный экземпляр нужно где-то хранить $\Rightarrow$ статическая переменная / статическое поле

Паттерн синглтон (singleton)

- Нельзя позволять пользователю самому создавать объект $\Rightarrow$ конструктор должен быть приватным, копирование нужно запретить
- Сам единственный экземпляр нужно где-то хранить $\Rightarrow$ статическая переменная / статическое поле
- Нужен доступ к этому экземпляру снаружи $\Rightarrow$ статические метод, возвращающий ссылку на экземпляр

Паттерн синглтон (singleton)

- Классический *синглтон Майерса*:

```
class Settings {  
private:  
    // Приватный конструктор  
    Settings();  
  
public:  
    // Запрет копирования  
    Settings(const Settings&) = delete;  
    Settings& operator=(const Settings&) = delete;  
  
    static Settings& instance() {  
        static Settings settings;  
        return settings;  
    }  
};
```

Паттерн синглтон (singleton)

- Классический *синглтон Майерса*:

```
int main() {  
    Settings::instance().loadFromFile(...);  
    ...  
}
```

Шаблонный синглтон Майерса

- Синглтонов может быть много, не хочется писать один и тот же код много раз

Шаблонный синглтон Майерса

- Синглтонов может быть много, не хочется писать один и тот же код много раз
- Сделаем шаблонный класс:

```
template <typename T>
class Singleton {
private:
    Singleton();

public:
    Singleton(const Singleton&) = delete;
    Singleton& operator=(const Singleton&) = delete;

    static T& instance() {
        static T singleton;
        return singleton;
    }
};
```

Шаблонный синглтон Майерса

- В шаблонном классе в этом подходе смысла уже мало, можно заменить на шаблонную функцию:

```
template <typename T>
T& singleton() {
    static T object;
    return object;
}
```

Шаблонный синглтон Майерса

```
class Settings {  
private:  
    ...  
public:  
    void loadFromFile(...) {...}  
};  
  
int main() {  
    singleton<Settings>().loadFromFile(...);  
}
```

Шаблонный синглтон Майерса

```
class Settings {  
private:  
    ...  
public:  
    void loadFromFile(...) {...}  
};  
  
int main() {  
    singleton<Settings>().loadFromFile(...);  
}
```

- Класс `Settings` опять можно создавать и копировать самому

Шаблонный синглтон Майерса

```
class Settings {  
private:  
    ...  
public:  
    void loadFromFile(...) {...}  
};  
  
int main() {  
    singleton<Settings>().loadFromFile(...);  
}
```

- Класс `Settings` опять можно создавать и копировать самому
- $\implies$ Отнаследуем `Settings` от `Singleton<Settings>`

Curiously Recurring Template Pattern (CRTP)

```
class Settings : public Singleton<Settings> {  
private:  
    friend class Singleton<Settings>;  
    Settings() = default;  
public:  
    void loadFromFile(...) {...}  
};
```

Curiously Recurring Template Pattern (CRTP)

```
class Settings : public Singleton<Settings> {  
private:  
    friend class Singleton<Settings>;  
    Settings() = default;  
public:  
    void loadFromFile(...) {...}  
};
```

- Класс `Settings` нельзя создать самому, только использовать через `Singleton<Settings>`

Curiously Recurring Template Pattern (CRTP)

```
class Settings : public Singleton<Settings> {  
private:  
    friend class Singleton<Settings>;  
    Settings() = default;  
public:  
    void loadFromFile(...) {...}  
};
```

- Класс `Settings` нельзя создать самому, только использовать через `Singleton<Settings>`
- Приём наследования класса `class A : public Base<A>` от шаблонного класса, у которого в качестве шаблонного аргумента передан тот же класс, называется CRTP – *Curiously Recurring Template Pattern*

- Пусть, в программе есть две функции: `f()` и `g()`

- Пусть, в программе есть две функции: `f()` и `g()`
- Если `f()` внутри вызывает `g()`, говорят, что `f()` *зависит от* `g()`

```
void f() {  
    ...  
    g();  
    ...  
}
```

- Аналогично, если в программе есть два класса: **A** и **B**

- Аналогично, если в программе есть два класса: A и B
- Если класс A использует/содержит класс B, говорят, что A *зависит от* B

```
class A {  
private:  
    B* b;  
public:  
    ...  
};
```


- Почему полезно думать про зависимости кода?

- Почему полезно думать про зависимости кода?
- Если мы хотим в новую библиотеку/программу/подсистему добавить некую функцию/класс, то нужно добавить и все его зависимости

- Почему полезно думать про зависимости кода?
- Если мы хотим в новую библиотеку/программу/подсистему добавить некую функцию/класс, то нужно добавить и все его зависимости
- $\implies$ Хочется, чтобы зависимостей было меньше

- Почему полезно думать про зависимости кода?
- Если мы хотим в новую библиотеку/программу/подсистему добавить некую функцию/класс, то нужно добавить и все его зависимости
- $\implies$ Хочется, чтобы зависимостей было меньше
- При этом, в зависимостях часто лежит переиспользуемый, обобщённый код

- Почему полезно думать про зависимости кода?
- Если мы хотим в новую библиотеку/программу/подсистему добавить некую функцию/класс, то нужно добавить и все его зависимости
- $\implies$ Хочется, чтобы зависимостей было меньше
- При этом, в зависимостях часто лежит переиспользуемый, обобщённый код
- $\implies$ Хочется, чтобы зависимостей было больше :)

Циклические зависимости

- Если две части программы (функции/классы/файлы/etc) зависят друг от друга, говорят о *циклической зависимости*

Циклические зависимости

- Если две части программы (функции/классы/файлы/etc) зависят друг от друга, говорят о *циклической зависимости*
- Это почти всегда плохо: эти части нельзя использовать по отдельности (независимо), возникают проблемы с порядком определения, и т.п.

```
// Приходится делать forward declaration,  
// иначе поле A::b имеет неизвестный тип  
class B;  
  
class A {  
private:  
    B* b;  
};  
  
class B {  
private:  
    A* a;  
};
```

- Часто циклическую зависимость можно устранить

Циклические зависимости

- Часто циклическую зависимость можно устранить
- Объединить созависимые компоненты в одну



Циклические зависимости

- Часто циклическую зависимость можно устранить
- Объединить созависимые компоненты в одну



- Выделить общую часть, от которой будут зависеть эти компоненты



Пример архитектуры программы: Гомоку

- Задача: спроектировать и реализовать игру гомоку (как крестики-нолики, 5 в ряд, на доске 15×15)

Пример архитектуры программы: Гомоку

- Задача: спроектировать и реализовать игру гомоку (как крестики-нолики, 5 в ряд, на доске 15×15)
- Спроектировать = продумать файлы/функции/классы/модули и их взаимодействие

Пример архитектуры программы: Гомоку

- Задача: спроектировать и реализовать игру гомоку (как крестики-нолики, 5 в ряд, на доске 15×15)
- Спроектировать = продумать файлы/функции/классы/модули и их взаимодействие
- Избежать излишних зависимостей (особенно циклических) между компонентами программы

Пример архитектуры программы: Гомоку

- Задача: спроектировать и реализовать игру гомоку (как крестики-нолики, 5 в ряд, на доске 15×15)
- Спроектировать = продумать файлы/функции/классы/модули и их взаимодействие
- Избежать излишних зависимостей (особенно циклических) между компонентами программы
- Реализовать автоматическое тестирование (т.н. *автотесты*)

- Компонента *Model*
 - Хранение внутреннего представления данных программы
 - Контроль правил игры и реализация ходов игроков
 - Обеспечение инвариантов внутреннего состояния

- Компонента *Model*
 - Хранение внутреннего представления данных программы
 - Контроль правил игры и реализация ходов игроков
 - Обеспечение инвариантов внутреннего состояния
- Компонента *View*
 - Взаимодействие с пользователем
 - Ввод ходов пользователя
 - Вывод состояния на экран

- Компонента *Model*
 - Хранение внутреннего представления данных программы
 - Контроль правил игры и реализация ходов игроков
 - Обеспечение инвариантов внутреннего состояния
- Компонента *View*
 - Взаимодействие с пользователем
 - Ввод ходов пользователя
 - Вывод состояния на экран
- Кто от кого должен зависеть?

Архитектура программы: паттерн Model-View

- Компонента *Model*
 - Хранение внутреннего представления данных программы
 - Контроль правил игры и реализация ходов игроков
 - Обеспечение инвариантов внутреннего состояния
- Компонента *View*
 - Взаимодействие с пользователем
 - Ввод ходов пользователя
 - Вывод состояния на экран
- Кто от кого должен зависеть? Подумаем о повторном использовании кода

Архитектура программы: паттерн Model-View

- Переиспользование *Model* – напомним разные *View*, использующие один *Model*:
 - Текстовый/графический интерфейс под разные платформы
 - Веб-интерфейс
 - Мобильное приложение

Архитектура программы: паттерн Model-View

- Переиспользование *Model* – напомним разные *View*, использующие один *Model*:
 - Текстовый/графический интерфейс под разные платформы
 - Веб-интерфейс
 - Мобильное приложение
- Переиспользование *View* – напомним универсальный *View*, работающий с разными моделями игр *Model*:
 - Гомоку
 - Шашки
 - Шахматы
 - Го

Архитектура программы: паттерн Model-View

- Переиспользование *Model* – напомним разные *View*, использующие один *Model*:
 - Текстовый/графический интерфейс под разные платформы
 - Веб-интерфейс
 - Мобильное приложение
- Переиспользование *View* – напомним универсальный *View*, работающий с разными моделями игр *Model*:
 - Гомоку
 - Шашки
 - Шахматы
 - Го
- Скорее всего, разные игры требуют и разного ввода/вывода, так что вариант с переиспользованием *View* не реалистичен

Архитектура программы: паттерн Model-View

- Переиспользование *Model* – напомним разные *View*, использующие один *Model*:
 - Текстовый/графический интерфейс под разные платформы
 - Веб-интерфейс
 - Мобильное приложение
- Переиспользование *View* – напомним универсальный *View*, работающий с разными моделями игр *Model*:
 - Гомоку
 - Шашки
 - Шахматы
 - Го
- Скорее всего, разные игры требуют и разного ввода/вывода, так что вариант с переиспользованием *View* не реалистичен
- $\implies$ Сделаем переиспользуемый *Model*, и от него будет зависеть *View*

- Model-View-Controller (MVC)
- Model-View-Presenter (MVP)
- Model-View-ViewModel (MVVM)

- Что входит в состояние игры?

- Что входит в состояние игры?
 - Чей сейчас ход / чем закончилась игра

- Что входит в состояние игры?
 - Чей сейчас ход / чем закончилась игра
 - Состояние всех клеток

- Что входит в состояние игры?
 - Чей сейчас ход / чем закончилась игра
 - Состояние всех клеток

```
enum class GameState {  
    XPlayerTurn, // ход крестиков  
    OPlayerTurn, // ход ноликов  
    XPlayerWin,  // крестики выиграли  
    OPlayerWin,  // нолики выиграли  
    Draw,        // ничья  
};  
  
// 'O' или 'X'  
using Player = char;
```

```
Player currentPlayer(GameState state) {  
    switch (state) {  
        case GameState::XPlayerTurn: return 'X';  
        case GameState::OPlayerTurn: return 'O';  
        default: return 0;  
    }  
}  
  
GameState switchPlayer(GameState state) {  
    switch (state) {  
        case GameState::XPlayerTurn: return GameState::OPlayerTurn;  
        case GameState::OPlayerTurn: return GameState::XPlayerTurn;  
        default: state;  
    }  
}
```

Гомоку: Model

```
class Model {
private:
    char board_[15][15];
    GameState state_ = GameState::XPlayerTurn;

    // После хода по координатам (x, y) проверить,
    // не окончена ли игра
    void checkWinner(int x, int y);

public:
    // Проинициализировать пустое поле
    Model();

    // Сделать ход и обновить состояние,
    // либо сообщить, что ход неверный
    bool move(int x, int y);
    char get(int x, int y) const;

    // Проверить, закончена ли игра
    bool finished() const;
    // Сообщить, есть ли победитель
    std::optional<Player> winner() const;
};
```

```
bool Model::move(int x, int y) {  
    // Игра уже окончена  
    if (finished())  
        return false;  
  
    // Клетка уже заполнена  
    if (board_[x][y] != ' ')  
        return false;  
  
    board_[x][y] = currentPlayer(state_);  
  
    checkWinner(x, y);  
  
    if (!finished())  
        state_ = switchPlayer(state_);  
}
```

```
class View {  
private:  
    Model& model_;  
  
public:  
    View(Model& model);  
  
    void processInput(std::istream& in, std::ostream& out);  
    void printBoard(std::ostream& out);  
    void showWinner(std::ostream& out);  
};
```

```
void View::processInput(std::istream& in, std::ostream& out) {
    while (true) {
        int x, y;
        in >> x >> y;
        if (model_.move(x, y))
            break;
        out << "Invalid move" << std::endl;
    }
}

void View::printBoard(std::ostream& out) {
    for (int y = 0; y < 15; ++y) {
        for (int x = 0; x < 15; ++x) {
            out << model_.get(x, y);
        }
        out << '\n';
    }
    out << std::flush;
}
```



```
int main() {  
    Model model;  
    View view(model);  
    while (!model.finished()) {  
        view.printBoard(std::cout);  
        view.processInput(std::cin, std::cout);  
    }  
    view.showWinner(std::cout);  
}
```

- Любой программный продукт тестируется специально обученными людьми

- Любой программный продукт тестируется специально обученными людьми
- Часто, процесс тестирования можно (и нужно!) автоматизировать:

- Любой программный продукт тестируется специально обученными людьми
- Часто, процесс тестирования можно (и нужно!) автоматизировать:
 - Проверить, что функция на известных входах даёт известные выходы

- Любой программный продукт тестируется специально обученными людьми
- Часто, процесс тестирования можно (и нужно!) автоматизировать:
 - Проверить, что функция на известных входах даёт известные выходы
 - Проверить, что сохраняются инварианты класса

- Любой программный продукт тестируется специально обученными людьми
- Часто, процесс тестирования можно (и нужно!) автоматизировать:
 - Проверить, что функция на известных входах даёт известные выходы
 - Проверить, что сохраняются инварианты класса
 - Проверить, что типовые сценарии приводят к ожидаемому результату

- Любой программный продукт тестируется специально обученными людьми
- Часто, процесс тестирования можно (и нужно!) автоматизировать:
 - Проверить, что функция на известных входах даёт известные выходы
 - Проверить, что сохраняются инварианты класса
 - Проверить, что типовые сценарии приводят к ожидаемому результату
 - Проверить, что правильно обрабатываются некорректные входные данные

- Любой программный продукт тестируется специально обученными людьми
- Часто, процесс тестирования можно (и нужно!) автоматизировать:
 - Проверить, что функция на известных входах даёт известные выходы
 - Проверить, что сохраняются инварианты класса
 - Проверить, что типовые сценарии приводят к ожидаемому результату
 - Проверить, что правильно обрабатываются некорректные входные данные
 - Проверить, что после очередных изменений в коде ничего не сломалось

- Любой программный продукт тестируется специально обученными людьми
- Часто, процесс тестирования можно (и нужно!) автоматизировать:
 - Проверить, что функция на известных входах даёт известные выходы
 - Проверить, что сохраняются инварианты класса
 - Проверить, что типовые сценарии приводят к ожидаемому результату
 - Проверить, что правильно обрабатываются некорректные входные данные
 - Проверить, что после очередных изменений в коде ничего не сломалось
- $\implies$ *Автоматические тесты (автотесты)*

- Тест – это отдельная программа, выполняющая тестирование

- Тест – это отдельная программа, выполняющая тестирование
- Часто все сценарии тестирования собраны в одну тестирующую программу

- Тест – это отдельная программа, выполняющая тестирование
- Часто все сценарии тестирования собраны в одну тестирующую программу
- Каждый сценарий – отдельная функция, проверяющая конкретное поведение

- Тест – это отдельная программа, выполняющая тестирование
- Часто все сценарии тестирования собраны в одну тестирующую программу
- Каждый сценарий – отдельная функция, проверяющая конкретное поведение
- Есть множество библиотек для автотестирования, упрощающих этот процесс: GoogleTest, Boost.Test, Catch2, doctest, QuickCheck

```
bool testSameMove() {  
    Model model;  
    // Делаем один и тот же ход два раза  
    model.move(0, 0);  
    // Второй раз move должна вернуть false  
    if (model.move(0, 0)) {  
        // Тест не прошёл  
        return false;  
    }  
    // Тест прошёл  
    return true;  
}
```

```
bool testHorizontalWin() {  
    Model model;  
    // Делаем простые ходы  
    for (int i = 0; i < 4; ++i) {  
        // Ход крестиков  
        model.move(i, 0);  
        // Ход ноликов  
        model.move(i, 1);  
    }  
    // Победный ход крестиков  
    model.move(4, 0);  
  
    // Условие прохождения теста  
    return model.finished() && model.winner() == 'X';  
}
```

Автотестирование

```
using TestFunction = bool (*)();

struct TestData {
    TestFunction function;
    const char* name;
};

class TestRunner {
private:
    std::vector<TestData> tests_;

public:
    void add(TestFunction function, const char* name);
    void run();
};

int main() {
    TestRunner runner;
    runner.add(testSameMove, "testSameMove");
    runner.add(testHorizontalWin, "testHorizontalWin");
    runner.run();
}
```



```
void TestRunner::run() {  
    int success = 0;  
  
    for (const TestData& test : tests_) {  
        if (test.function()) {  
            std::cout << "Test " << test.name << ": success\n";  
            ++success;  
        } else {  
            std::cout << "Test " << test.name << ": failure\n";  
        }  
    }  
  
    std::cout << "Success: " << success << "/"  
              << tests_.size() << "\n";  
}
```